МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»

Інститут прикладної математики та фундаментальних наук

Кафедра прикладної математики

Звіт про виконання лабораторної роботи № 1
з дисципліни «Автоматизовані системи тестування програмного продукту»

На тему:
«МОДУЛЬНЕ ТЕСТУВАННЯ ПЗ
(бібліотека AnalaizerClassLibrary, метод Format)»

Варіант № 4

Виконав:
студент групи ПМ-31
Погорілець Д. В.
Здано: 05.10.2026

Перевірив:
доцент каф. ПМ
Пабирівський В. В.
Підпис: _____

Львів – 2026

Постановка завдання

За варіантом 4 потрібно написати набір модульних тестів з максимальним покриттям коду для методу Format бібліотеки AnalaizerClassLibrary. Набір вхідних даних має зберігатися в базі даних, структуру якої треба спроєктувати під варіант завдання.

Роботу виконано у середовищі MS Visual Studio. Використано MS SQL Server Express LocalDB та фреймворк MSTest.

Вихідний код і скрипти бази даних розміщено у репозиторії GitHub:
https://github.com/dmytropohorol/LPNUTesting/tree/main/1Lab

Короткі теоретичні відомості

Модульний тест перевіряє найменшу самостійну частину програми окремо від решти системи. Тест складається з трьох фаз: підготовка стану, виклик тестованого коду, перевірка результату. Для C# найпоширеніші фреймворки MSTest, NUnit та xUnit.

Тест, керований даними, бере вхідні значення із зовнішнього джерела, а не з коду. Логіка перевірки відокремлена від самих даних, тому набір даних нарощується без зміни коду. У MSTest для цього призначений атрибут DynamicData.

Повноту набору тестів оцінюють покриттям коду. Покриття операторів показує частку виконаних рядків, покриття гілок показує частку умов, які набули обох значень. Для методу Format обрано саме ці два критерії. Покриття шляхів недосяжне, бо метод містить цикли по рядку довільної довжини.

Тестові дані підібрано методом класів еквівалентності та аналізом граничних значень. Вхідні дані розбито на групи з однаковою поведінкою програми, з кожної групи взято представника, а також значення на межах груп.

Хід роботи

Аналіз методу Format

Метод не має параметрів. Він читає та перезаписує публічне статичне поле expression класу AnalaizerClass. Повертає або відформатований вираз, або повідомлення про помилку, що починається зі знака амперсанда. Через роботу зі спільним статичним полем тести не можна виконувати паралельно, тому у збірці тестів встановлено атрибут DoNotParallelize.

Метод послідовно виконує такі перевірки: видаляє всі пробіли, перевіряє довжину виразу, повертає порожній рядок для порожнього входу, шукає невідомі символи, перевіряє

перший та останній символи, а потім у циклі перевіряє пари сусідніх символів. Усього метод має 14 гілок виходу, і на кожну з них створено окремий клас тестових даних.

Баланс та глибину вкладеності дужок, діапазон чисел і сумарну кількість операндів метод Format не перевіряє. Це робота методів CheckCurrency, CalcClass та RunEstimate.

Проєктування бази даних

Спроєктовано базу даних CalcTestDb із трьох таблиць. Таблиця ErrorCode є довідником кодів помилок зі специфікації. Таблиця TestCategory описує класи еквівалентності та прив'язує кожен з них до конкретної гілки методу. Таблиця FormatTestCase містить 115 тестових випадків.

У базі зберігається ім'я константи помилки, а не її текст. Повний текст тест бере з класу ErrorsExpression через рефлексію, тому повідомлення має одне джерело правди.

Вхідний вираз заданий парою значень InputPattern та InputRepeat. Вираз будується як повторення фрагмента, тому граничні випадки довжиною 65536 та 65537 символів займають у таблиці по одному рядку.

Обмеження цілісності на рівні бази не дозволяє задати для одного випадку одночасно очікувану помилку та очікуваний результат.

Тестовий проєкт

До рішення додано проєкт AnalizerClassLibrary.Tests на платформі net472. Кожен рядок таблиці FormatTestCase перевіряється чотирма тестами: збіг результату зі значенням з бази, наявність знака амперсанда лише у повідомленнях про помилку, відсутність пробілів у полі expression після виклику, однаковий результат при повторному виклику. Разом із сімома тестами цілісності набору це дає 467 тестів.

Розгортання бази даних та прогін тестів

Базу даних розгортає скрипт Deploy-TestDatabase.ps1. Він створює схему, наповнює довідники та вносить тестові випадки. Результат розгортання показано у лістингу 1.

Лістинг 1 – Результат розгортання бази даних CalcTestDb

```
Сервер: (localdb)\MSSQLLocalDB
  -> 01_CreateSchema.sql
  -> 02_SeedReferenceData.sql
  -> 03_SeedFormatTestCases.sql

 Id  Категорія                          Тестів
```

```
 1  Empty                                3
 2  SpaceRemoval                         5
 3  ValidExpression                     27
 4  MaxLength                            4
 5  UnknownSymbol                       14
 6  InvalidStart                         7
 7  InvalidEnd                          11
 8  DigitFollowedByBracketOrUnary        7
 9  TwoOperatorsInRow                    7
10  OperatorFollowedByInvalid            3
11  OpenBracketFollowedByInvalid         6
12  CloseBracketFollowedByInvalid        5
13  UnaryFollowedByInvalid              10
14  CheckPrecedence                      6
    РАЗОМ                              115
```

Результат прогону всього набору тестів показано у лістингу 2.

Лістинг 2 – Результат прогону модульних тестів

```
Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed!  - Failed: 0, Passed: 467, Skipped: 0, Total: 467,
           Duration: 959 ms - AnalizerClassLibrary.Tests.dll (net472)
```

Той самий прогін у вікні Test Explorer середовища Visual Studio показано на рисунку 1. Видно, що чотири тестові методи розгорнулися у 460 тестів за кількістю рядків таблиці, а разом із тестами цілісності набору виконано 467 тестів.

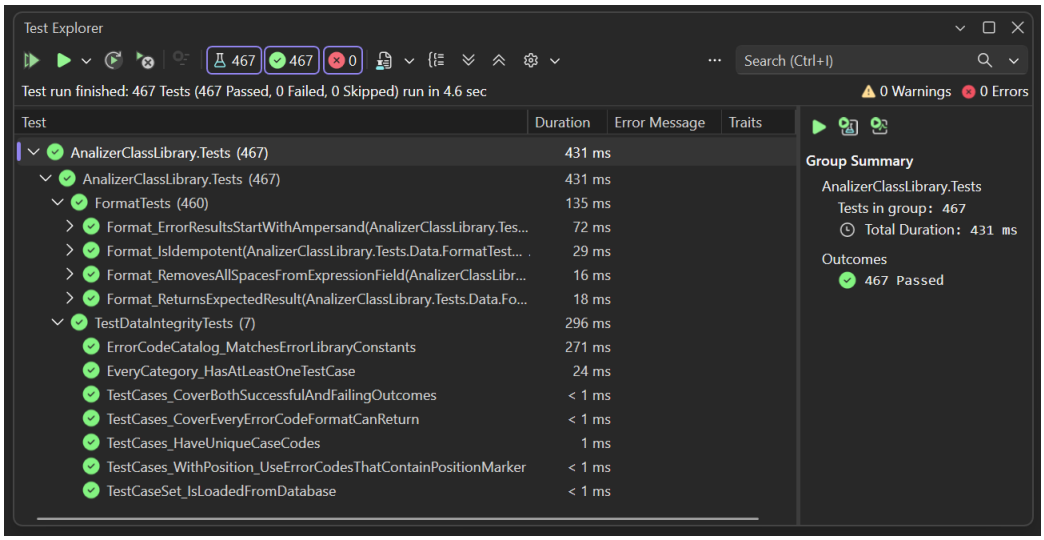


Рисунок 1 – Результат прогону тестів у вікні Test Explorer

Покриття коду

Покриття виміряно пакетом coverlet під час прогону. Результат по методах класу AnalaizerClass наведено у лістингу 3.

Лістинг 3 – Покриття коду класу AnalaizerClass

```
  CheckCurrency            рядки   0.00%   гілки   0.00%
  Format                   рядки  98.73%   гілки  98.71%
  IsOperator               рядки   0.00%   гілки   0.00%
  IsDelimeter              рядки   0.00%   гілки   0.00%
  GetPriority              рядки   0.00%   гілки   0.00%
  CreateStack              рядки   0.00%   гілки   0.00%
  RunEstimate              рядки   0.00%   гілки   0.00%
  InsertSymbol             рядки   0.00%   гілки   0.00%
  ReplaceUnaryPlusMinus    рядки   0.00%   гілки   0.00%
  Estimate                 рядки   0.00%   гілки   0.00%
```

Метод Format покрито на 98,73 відсотка за операторами, тобто 78 рядків із 79, та на 98,71 відсотка за гілками, тобто 77 із 78. Решта методів класу не покрита, бо за варіантом 4 тестується лише метод Format.

Непокритим залишився один оператор у блоці перевірки символів після унарної операції. Він повертає ERROR_05, коли унарна операція стоїть останнім символом виразу.

Цей оператор недосяжний. Щоб керування дійшло до нього, останнім символом виразу має бути m або p. Але перевірка останнього символу виконується раніше, ще до входу в цикл, і повертає ERROR_05 для будь-якого символу, що не є цифрою та не є закриваючою дужкою. Символи m і p під цю умову підпадають, тому метод завершує роботу до циклу.

Отже, непокритий оператор є мертвим кодом і дублює вже виконану перевірку. Покриття досяжного коду методу Format становить 100 відсотків.

Перевірка якості набору тестів

Щоб переконатися, що тести звіряють результат, а не проходять вхолосту, у базі даних навмисно змінено очікувані значення двох тестових випадків. Результат прогону показано у лістингу 4.

Лістинг 4 – Прогін тестів після спотворення двох рядків бази даних

```
Failed TC-03-03 [ValidExpression] Додавання
  Assert.AreEqual failed. Expected:<WRONG>. Actual:<2+3>.

Failed TC-05-02 [UnknownSymbol] Спецсимвол '&' усередині виразу
  Expected:<&Error 02 at <99> ...>  Actual:<&Error 02 at <1> ...>

Failed!  - Failed: 2, Passed: 465, Skipped: 0, Total: 467
```

Впали рівно ті два тести, яких стосувалася зміна. Після повторного розгортання бази всі 467 тестів знову проходять. Це підтверджує, що дані читаються з MS SQL Server і що перевірки не є тавтологічними.

Висновки

Виконано завдання варіанта 4. Розроблено набір із 467 модульних тестів для методу Format, усі вони проходять успішно.

Спроєктовано базу даних MS SQL Server із трьох таблиць, яка містить 115 тестових випадків у 14 класах еквівалентності. Кожен клас прив’язано до гілки методу, тому база одночасно є документацією покриття.

Тести, керовані даними, дозволили перевірити 115 наборів чотирма перевірками, маючи в коді лише чотири тестові методи. Новий тестовий випадок додається одним запитом INSERT, без зміни та перезбирання коду.

Досягнуто покриття 98,73 відсотка операторів і 98,71 відсотка гілок. Єдиний непокритий оператор доведено недосяжним, тому покриття досяжного коду становить 100 відсотків.

Крім самих помилок, тести виявили у методі Format мертвий код. Це показує, що модульні тести корисні не лише для контролю регресій, а й як засіб перевірки відповідності коду специфікації.